

Installation Instructions

UrbanTransit IQ is a two-tier application: a **FastAPI** Python backend over a Big-Data pipeline (Hadoop HDFS + Apache Spark / PySpark + Spark MLlib and an independent Python / scikit-learn / XGBoost pipeline) and a **React (Vite)** single-page dashboard. The stack follows the SRS software requirements.

Prerequisites

Tool	Version	Purpose
Python	3.12.x	pipeline, ML models, FastAPI backend
Node.js	18+	React frontend (Vite)
Java (JDK)	17	Apache Spark / HDFS (run under WSL/Linux)
Git	any	version control

Verify:

```
python --version      # 3.12.x (PySpark needs <=3.12, install from python.org)
node -v               # v18+
java -version          # 17.x (WSL/Linux for Spark)
```

1. Python dependencies (pipeline + backend)

```
pip install -r backend/requirements.txt
```

2. Frontend dependencies (React)

```
npm install
```

3. Email delivery (optional)

Verification / reset codes. Copy **config/email.env.example** to **config/email.env** and add Gmail App-Password creds for real email; or run the bundled local mail server (offline demo); or leave unset and codes appear on-screen + in reports/email_outbox.log.

4. Hadoop / HDFS + Spark (evidence machine — WSL/Linux)

Windows has no Java, so the Spark/HDFS layer runs under WSL2 Ubuntu (Java 17, Hadoop 3.4.1, PySpark 4.2). One-time setup and the four Spark jobs:

```
bash hdfs_scripts/wsl_setup_spark.sh      # venv + PySpark
bash hdfs_scripts/wsl_setup_hdfs.sh       # single-node HDFS + upload dataset
bash hdfs_scripts/wsl_run_spark_jobs.sh   # ingest -> quality -> features -> MLlib
bash hdfs_scripts/wsl_run_spark_on_hdfs.sh # Spark reading/writing hdfs://
```

The day-to-day pipeline also runs natively on Windows via the equivalent pandas/pyarrow engine (below); both produce identical data-quality counts.

5. Database

SQLite, auto-created at **database/urbantransit.db** by the seed script (no server). Delete the file to reset to a clean schema.

Execution Instructions

Run the pipeline once (top to bottom), then start the two servers.

A. Build data, models and app database

```
python data_generator/generate_dataset.py      # 2.06M-record dataset
python python_pipeline/process.py              # clean + join + features -> Parquet
python python_pipeline/train_models.py         # dual ML + forecast + clustering
python recommendation_engine/generate_recommendations.py
python database/seed.py                       # create DB + 4 demo accounts
```

B. Start the backend API (Swagger at /docs)

```
python -m uvicorn backend.src.main:app --port 8000
# http://localhost:8000/docs
```

C. Start the React dashboard

```
npm run dev
# http://localhost:5173
```

D. Run the tests

```
python -m pytest backend/tests -q           # 16 tests (data, models, auth, RBAC)
```

Login accounts (role-based access)

Account	Password	Access level
operator	operator123	Live operations: dashboards, delays, occupancy, recommendations, what-if
analyst	analyst123	+ Spark-vs-Python lab, data quality, reports, CSV export
evaluator	evaluator123	Read-everything jury account (all pages)
admin	admin123	Full control + HDFS/Spark monitor, audit, user management

New users may self-register; the account is verified by an emailed 6-digit code and is granted the Analyst role. Passwords are bcrypt-hashed; sessions use JWT; roles are enforced server-side.

GitHub Repository

Public repository (replace with your team's URL after the first push):

```
https://github.com/<your-team>/urbantransit-iq

# first push:
git init
git add .
git commit -m "UrbanTransit IQ - Big Data & Data Science transport analytics"
git branch -M main
git remote add origin https://github.com/<your-team>/urbantransit-iq.git
git push -u origin main
```

Large generated data (raw_data/, parquet_data/), node_modules/, the local database and secrets (config/email.env) are git-ignored; 1,000-row samples of every table are committed under sample_data/ and the dataset regenerates with the command above.